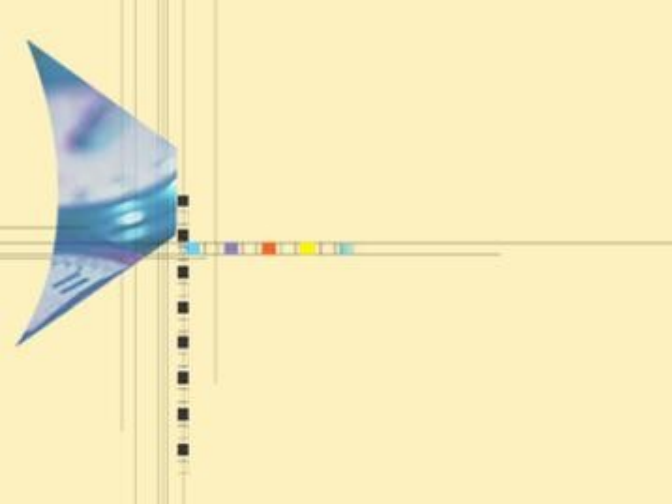




第 4 讲

动态规划

—A powerful method for combinatorial problems



主要内容

- 4.1 引言
- 4.2 动态规划策略的算法设计思想及分析方法
- 4.3 动态规划策略的经典应用（可选讲部分）



主要内容

4.1 引言

4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

4.1.2 最优化问题

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）

4.1.4 简单问题例算法的分析

4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

- 定义 Fibonacci 序列 f_n ，序列中的每一个数是它前面两个数的和：

$$f_1 = 1, f_2 = 1, f_3 = 2, f_4 = 3, f_5 = 5, f_6 = 8, f_7 = 13, \dots$$

- 我们来看这个序列的递归定义

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ f(n-1) + f(n-2) & \text{if } n \geq 3 \end{cases}$$

- 这个定义暗示一个看起来像下面那样表示的过程：

- :
 1. procedure $f(n)$
 2. if $(n=1)$ or $(n=2)$ then return 1
 3. else return $f(n-1)+f(n-2)$

? 传统的解决方法是什么？



4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

- 我们把递推式做适当的展开：

$$\begin{aligned}f(n) &= f(n-1) + f(n-2) \\&= 2f(n-2) + f(n-3) \\&= 3f(n-3) + 2f(n-4)\end{aligned}$$

递归算法的缺陷

- 这导致了巨大数量的重复调用。假设计算 $f(1)$ 和 $f(2)$ 需要一个单位时间，那么这个过程的时间复杂性可以表示为

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ T(n-1) + T(n-2) & \text{if } n \geq 3 \end{cases}$$

- 递推式的解是： $T(n) = f(n) \approx \Theta(\phi^n)$ $\phi = (1 + \sqrt{5})/2 \approx 1.618\ 03$
- 可以看出，计算 $f(n)$ 所需要的运行时间对于 n 的值是**指数**的，

? Much more efficient solution?

4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

一种有效算法

- 采取自底向上的方式递推求值，并把中间结果存储起来以便以后用来计算所要求的解。利用这种技术可以设计出特别有效的算法来解决许多组合最优化问题。
- 对于 Fibonacci 函数来说，如果从 f_1 开始自底向上地计算直至到 f_n ，只需要 $\Theta(n)$ 时间和 $\Theta(1)$ 空间。和上面的方法相比，可以很大程度地降低时间复杂度。

4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

- 二项式系数定义如下：

$$\binom{n}{k} = \begin{cases} 1 & \text{if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{if } 0 < k < n \end{cases}$$

- 与 Fibonacci 系数类似，递归算法的时间复杂度正比于其自身：

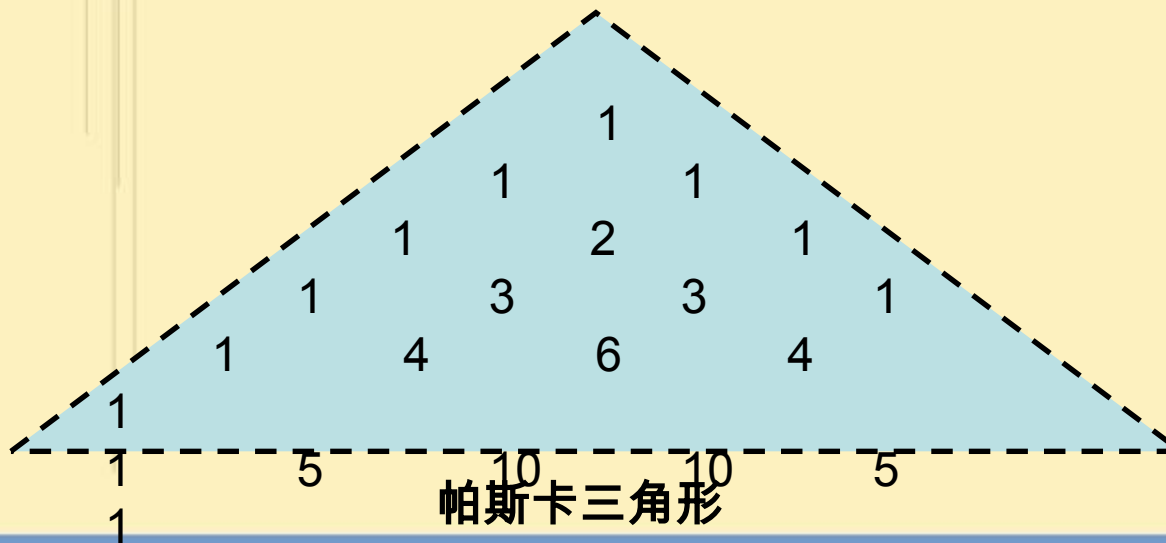
$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

此函数增长迅速。例如，当 $k=n/2$ （假设 n 是偶数），可知：

$$\binom{n}{n/2} = \frac{n!}{((n/2)!)^2} \approx \frac{\sqrt{2\pi n} n^n / e^n}{\pi n (n/2)^n / e^n} = \frac{2^n}{\sqrt{\pi n}}$$

4.1.1 递归表达式的归纳求解与迭代求解方法的复杂度对比分析（斐波那契序列、二项式系数）

- 一种有效计算 $\binom{n}{k}$ 的方法可以用按行构造帕斯卡三角来进行，
- 一旦计算出 $\binom{n}{k}$ 的值，计算就立即停止。



- 1、每个数等于它上方两数之和。
- 2、每行数字左右对称，由1开始逐渐变大。
- 3、第 n 行的数字有 n 项。
- 4、第 n 行数字和为 2^{n-1} 。



4.1.2 最优化问题

- 通常采用动态规划对问题进行**优化**。对于一个问题，可以有很多可能的解决方案。每个解决方案有一个值，我们希望找到一个最佳的（最小或最大）值对应的解决方案，我们称这样的解决办法为**最优的解决方案**。

4.1.2 最优化问题

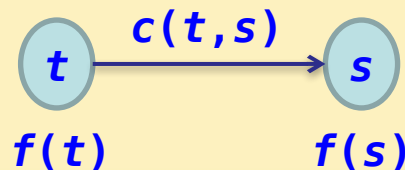
- **最优性原理**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优性原理。

$$f(A) = 0$$

$$f(s) = \underset{\text{存在 } \langle t, s \rangle \text{ 的有向边}}{\text{MIN}} \{f(t) + c(t, s)\}$$

$f(s)$ 表示初始状态 A 到状态 s 的最短路径长度

子问题的解也是最优的



4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）



问题描述：

- 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。要算出这 n 个矩阵的连乘积 $A_1 A_2 \dots A_n$ 。
- 由于矩阵乘法满足结合律，故计算矩阵的连乘积可以有許多不同的计算次序。这种计算次序可以用加括号的方式来确定。若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用 2 个矩阵相乘的标准算法计算出矩阵连乘积。
- 例如，矩阵连乘积 $A_1 A_2 A_3 A_4$ 有 5 种不同的完全加括号的方式

$(A_1 (A_2 (A_3 A_4)))$,
 $(A_1 ((A_2 A_3) A_4))$,
 $((A_1 A_2) (A_3 A_4))$,
 $((A_1 (A_2 A_3)) A_4)$,
 $((A_1 A_2) A_3) A_4$.

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）

- 每一种完全加括号的方式对应于一个矩阵连乘积的计算次序，这决定着作乘积所需要的计算量。
- 首先考虑两个矩阵相乘的成本。标准的算法由下面的伪代码给出：

```
Algorithm MATRIX-MULTIPLY(A, B)
if columns[A]  $\neq$  rows[B]
then error "incompatible dimensions"
else for  $i \leftarrow 1$  to rows[A] //p
      for  $j \leftarrow 1$  to columns[B] //r
        C[i, j]  $\leftarrow$  0
        for  $k \leftarrow 1$  to columns[A] //p
          C[i, j]  $\leftarrow$  C[i, j] + A[i, k]
          B[k, j]
        end for
      end for
    end for
  end if
return C
```

相乘的两个矩阵必须是兼容的

(**compatible**) : 即 A 矩阵的列数必须等于 B 矩阵的行。若 A 是一个 $p \times q$ 矩阵，B 是一个 $q \times r$ 矩阵，则计算其乘积 $C=AB$ 的标准算法中，需要进行 pqr 次数乘。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列)

- 先考察 3 个矩阵 $\{A_1, A_2, A_3\}$ 连乘的情况。设这三个矩阵的维数分别为 10×100 , 100×5 , and 5×50 。

加括号方式	第一次相乘次数	第二次相乘次数	总相乘次数
$((A_1 A_2)A_3)$	$10 \cdot 100 \cdot 5 = 5000$	$10 \cdot 5 \cdot 50 = 2500$	7500
$(A_1(A_2 A_3))$	$100 \cdot 5 \cdot 50 = 25000$	$10 \cdot 100 \cdot 50 = 50000$	75000

? 问题

如何确定计算矩阵连乘积 $A_1 A_2 \dots A_n$ 的计算次序 (完全加括号方式) , 使得依此次序计算矩阵连乘积需要的数乘次数最少。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 蛮力方法：穷举

基本思路

- 对于 n 个矩阵的连乘积，设其不同的计算次序为 $P(n)$
- 当 $n = 1$ ，只有一个矩阵，因此 $P(n)=1$
- 当 $n \geq 2$ ，对于前 k 个矩阵有 $P(k)$ 种方法放置括号。对于 $P(k)$ 中的每一种方法，可对余下的 $P(n-k)$ 个矩阵放置括号，总共有 $P(k)P(n-k)$ 种方法。由于可以假设 k 是 1 到 $n-1$ 中的任意值，对于 n 个矩阵放置括号的所有方法数由下面的和式给出：

$$P(n) = \begin{cases} 1 & \text{if } n = 1. \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 蛮力方法：穷举

- 递推式的解 p133
$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi n}^{1.5}}$$
- 由于对于每个括弧号表达式，找到数量乘法次数的时间耗费是 $\Theta(n)$ ，这样用蛮力方法可以求得找到 n 个矩阵相乘的最优方法所需的运行时间是 $\Omega(4^n/n^{0.5})$ ，它甚至对于一个中等规模的 n 值也是不切实际的。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最优子结构

- 将这个计算次序在矩阵 $A(k)$ 和 $A(k+1)$ 之间将矩阵链断开, 则生成两个矩阵链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 。计算 $A_1 \dots A_n$ 的最优次序所包含的计算矩阵子链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 的次序也是最优的。
- 矩阵子链的所耗费的计算量越低, 则矩阵链 $A_1 \dots A_n$ 所耗费的计算量越低
- 总共的计算量包括 $\text{cost}(A_1 \dots A_k) + \text{cost}(A_{k+1} \dots A_n) + \text{cost}$ (两个矩阵子链的乘积生成的矩阵相乘)。而无论子问题的解决方案如何, 最后一项的计算量不变。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最优子结构

- 将矩阵连乘积 $A(i)A(i+1)\dots A(j)$ 简记为 $A[i:j]$, 设计算 $A[i:j](1 \leq i \leq j \leq n)$ 所需要的最少乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
- 当 $i = j$ 时, $A[i:j]=A_i$, 因此, $m[i,i] = 0$, $i = 1,2,\dots,n$
- 当 $i < j$ 时, $m[i,j] = m[i,k] + m[k+1,j] + p(i-1)p(k)p(j)$
 - 这里 $A(i)$ 的维数为 $p(i-1)*p(i)$ (注: $p(i-1)$ 为矩阵 $A(i)$ 的行数, $p(i)$ 为矩阵 $A[i]$ 的列数)
- 可以递归地定义 $m[i,j]$ 为:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p[i - 1]p[k]p[j]\} & \text{if } i < j \end{cases}$$

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 伪代码

- 下面的伪代码假定矩阵 A_i 尺寸 $p(i-1) \times p(i)$ for $i = 1, 2, \dots, n$.
- 输入是 $p = p[0], p[1], \dots, p[n]$, 其中 $length[p] = n + 1$.
- 该过程使用的辅助表 $m[1..n, 1..n]$ 用于存储的最小 $m[i, j]$ 的计算耗费
- 辅助表 $s[1..n, 1..n]$ 记录索引 k 达到最佳的成本计算时的 $m[i, j]$, 我们将使用表 S 构建一个最优方案。

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）— 伪代码

Algorithm MATRIX-CHAIN-ORDER(p)

$n \leftarrow \text{length}[p] - 1$ // n 个矩阵

for $i \leftarrow 1$ **to** n

$m[i, i] \leftarrow 0$

end for

for $l \leftarrow 2$ **to** n $\triangleright l$ is the chain length.

for $i \leftarrow 1$ **to** $n - (l - 1)$

$j \leftarrow i + (l - 1)$

$m[i, j] \leftarrow \infty$

for $k \leftarrow i$ **to** $j - 1$

$q \leftarrow m[i, k] + m[k + 1, j] + p[i-1] p[k] p[j]$ // 矩阵乘法次数

if $q < m[i, j]$ // 最小数乘次数

then $m[i, j] \leftarrow q, s[i, j] \leftarrow k$

end if

end for

end for

end for

return m and s

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）— 例子

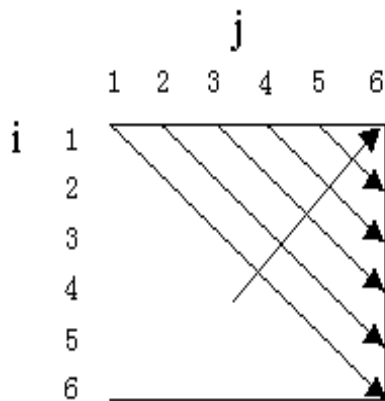
The length of chain is 5, and the dimensions is:

A_1 5×10 , A_2 10×4 , A_3 4×6 , A_4 6×10 , A_5 10×2

$m[1,1]=0$	$m[1,2]=200$	$m[1,3]=320$	$m[1,4]=620$	$m[1,5]=348$
	$m[2,2]=0$	$m[2,3]=240$	$m[2,4]=640$	$m[2,5]=248$
		$m[3,3]=0$	$m[3,4]=240$	$m[3,5]=168$
			$m[4,4]=0$	$m[4,5]=120$
				$m[5,5]=0$

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）— 例子

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

	j	1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

	j	1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 算法分析

- 算法 matrixChain 的主要计算量取决于算法中对 l , i 和 k 的 3 重循环。循环体内的计算量为 $O(1)$, 而 3 重循环的总次数为 $O(n^3)$ 。
- 算法的时间复杂性是 $\Omega(n^3)$. 算法的空间复杂度是 $\Theta(n^2)$ 用于存储 m 和 s 表。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最长公共子序列

- 若给定序列 $X = \{x_1, x_2, \dots, x_m\}$, 则另一序列 $Z = \{z_1, z_2, \dots, z_k\}$, Z 是 X 的 **子序列** 是指存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$ 有:
 $z_j = x_{i_j}$ 。
- 例如, 序列 $Z = \{B, C, D, B\}$ 是序列 $X = \{A, B, C, B, D, A, B\}$ 的子序列, 相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。
- 给定 2 个序列 X 和 Y , 当另一序列 Z 既是 X 的子序列又是 Y 的子序列时, 称 Z 是序列 X 和 Y 的 **公共子序列**。
- 给定 2 个序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$, 找出 X 和 Y 的 **最长公共子序列**。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最长公共子序列

- 在字母表 E 上，分别给出两个长度为 n 和 m 的字符串 A 和 B ，确定在 A 和 B 中最长公共子序列的长度。
- 如果 $\Sigma = \{x, y, z\}$ ， $A = zxyxyz$ 和 $B = xy yzx$ ，那么 xyy 同时是 A 和 B 的长度为 3 的子序列。然而，它不是 A 和 B 最长的公共子序列，因为字符串 $xyyz$ ，也是 A 和 B 公共的子序列，由于这两个字符串没有比 4 更长的公共子序列，因此， A 和 B 的最长的公共子序列的长度是 4。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最长公共子序列

? 问题描述

- 在最长公共子序列问题中 (**longest-common-subsequence (LCS) problem**) , 给定两个序列 $A = a_1, a_2, \dots, a_m$ and $B = b_1, b_2, \dots, b_n$, 并希望找到 A 和 B 中的最长公共子序列以及序列长度。 .

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 蛮力搜索算法

- 列举 A 所有的 2^n 个子序列
- 对于每一个子序列在 $O(m)$ 时间内来确定它是否也是 B 的子序列
- 很明显, 此算法的时间复杂性是 $O(m2^n)$, 是指数复杂性的。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 基本思想

- 设序列 $A=\{a_1,a_2,\dots,a_n\}$ 和 $B=\{b_1,b_2,\dots,b_m\}$ 的最长公共子序列为 $C=\{c_1,c_2,\dots,c_k\}$, 则
 - (1) 若 $a_n=b_m$, 则 $c_k=a_n=b_m$, 且 C_{k-1} 是 A_{n-1} 和 B_{m-1} 的最长公共子序列。
 - (2) 若 $a_n \neq b_m$ 且 $c_k \neq a_n$, 则 C 是 A_{n-1} 和 B 的最长公共子序列。
 - (3) 若 $a_n \neq b_m$ 且 $c_k \neq b_m$, 则 C 是 A 和 B_{m-1} 的最长公共子序列。
- 由此可见, 2 个序列的最长公共子序列包含了这 2 个序列的前缀的最长公共子序列。因此, 最长公共子序列问题具有最优子结构性质。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 最优子结构

- 设序列 $A=\{a_1, a_2, \dots, a_n\}$ 和 $B=\{b_1, b_2, \dots, b_m\}$, 令 $L[i, j]$ 表示 A 和 B 的最长公共子序列的 **长度**。

- 观察结论 7.1

如果 i 和 j 都大于 0 , 那么

- 若 $a_i = b_j$, $L[i, j] = L[i - 1, j - 1] + 1$;
- 若 $a_i \neq b_j$, $L[i, j] = \max\{L[i, j - 1], L[i - 1, j]\}$ 。

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 伪代码

算法 7.1 LCS

输入：字母表 Σ 上的两个字符串 A 和 B，长度分别为 n 和 m

输出：A 和 B 最长公共子序列的长度。

1. **for** $i \leftarrow 0$ **to** n // $j=0$ 时, $L[i,j]=0$
2. $L[i,0] \leftarrow 0$
3. **end for**
4. **for** $j \leftarrow 0$ **to** m // $i=0$ 时, $L[i,j]=0$
5. $L[0,j] \leftarrow 0$
6. **end for**
7. **for** $i \leftarrow 0$ **to** n
8. **for** $j \leftarrow 0$ **to** m
9. **if** $a_i = b_j$ **then** $L[i,j] \leftarrow L[i-1,j-1] + 1$
10. **else** $L[i,j] \leftarrow \max\{L[i,j-1], L[i-1,j]\}$
11. **end if**
12. **end for**
13. **end for**
14. **return** $L[n,m]$

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）— 例子

- Example: $X = A, B, C, B, D, A, B$ and $Y = B, D, C, A, B, A$.

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
i	x_i								
0			0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖1	←1	↖1
2	B		0	↖1	←1	←1	↑1	↖2	←2
3	C		0	↑1	↑1	↖2	←2	↑2	↑2
4	B		0	↖1	↑1	↑2	↑2	↖3	←3
5	D		0	↑1	↖2	↑2	↑2	↑3	↑3
6	A		0	↑1	↑2	↑2	↖3	↑3	↖4
7	B		0	↖1	↑2	↑2	↑3	↖4	↑4

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 伪代码

Algorithm LCS-LENGTH(X, Y)

```
 $m \leftarrow \text{length}[X]$ 
 $n \leftarrow \text{length}[Y]$ 
for  $i \leftarrow 0$  to  $m$ 
     $L[i, 0] \leftarrow 0$ 
end for
for  $j \leftarrow 0$  to  $n$ 
     $L[0, j] \leftarrow 0$ 
end for
for  $i \leftarrow 1$  to  $m$ 
    for  $j \leftarrow 1$  to  $n$ 
        if  $x_i = y_j$  then  $L[i, j] \leftarrow L[i - 1, j - 1] + 1$ ,  $b[i, j] \leftarrow "\nwarrow"$ 
        else if  $L[i - 1, j] \geq c[i, j - 1]$  then  $L[i, j] \leftarrow L[i - 1, j]$ ,  $b[i, j] \leftarrow "\uparrow"$ 
        else  $L[i, j] \leftarrow L[i, j - 1]$ ,  $b[i, j] \leftarrow " \leftarrow "$ 
        end if
    end for
end for
return  $L$  and  $b$ 
```

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 追踪解

算法 PRINT-LCS(b, X, i, j)

if $i = 0$ or $j = 0$ **then return**
end if

if $b[i, j] = "\nwarrow"$ **then** PRINT-LCS($b, X, i - 1, j - 1$), **print** x_i
else if $b[i, j] = "\uparrow"$ **then** PRINT-LCS($b, X, i - 1, j$)
else PRINT-LCS($b, X, i, j - 1$)
end if

4.1.3 简单问题例的算法描述 (矩阵链乘法、最长公共子序列) — 算法改进

- 在算法 LCS-LENGTH 和 PRINT-LCS 中, 可进一步将数组 **b** 省去。事实上, 数组元素 $L[i][j]$ 的值仅由 $L[i-1][j-1]$, $L[i-1][j]$ 和 $L[i][j-1]$ 这 3 个数组元素的值所确定。对于给定的数组元素 $L[i][j]$, 可以不借助于数组 b 而仅借助于 L 本身在时间内确定 $L[i][j]$ 的值是由 $L[i-1][j-1]$, $L[i-1][j]$ 和 $L[i][j-1]$ 中哪一个值所确定的。
- 如果只需要计算最长公共子序列的长度, 则算法的空间需求可大大减少。事实上, 在计算 $L[i][j]$ 时, 只用到数组 L 的第 i 行和第 $i-1$ 行。因此, **用 2 行的数组空间** 就可以计算出最长公共子序列的长度。进一步的分析还可将空间需求减至 $O(\min(m,n))$ 。

4.1.3 简单问题例的算法描述（矩阵链乘法、最长公共子序列）— 算法分析

- 最长公共子序列问题的最优解能够在
 $\square (nm)$ 时间和 $\square (\min\{m,n\})$ 空间内得到。

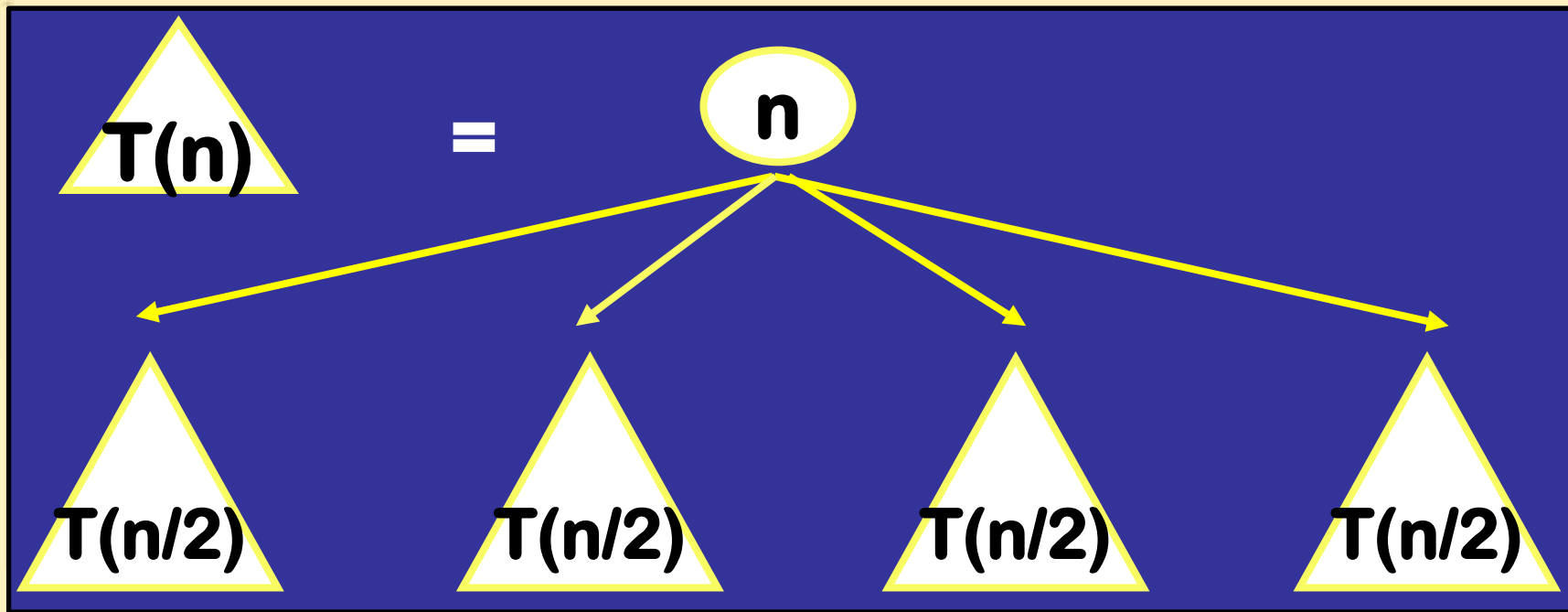


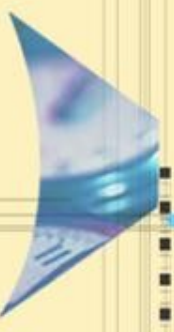
4.1.4 简单问题例算法的分析

- **动态规划**是一种解决多阶段决策问题的优化方法，把多阶段过程转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解。

4.1.4 简单问题例算法的分析

- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题。



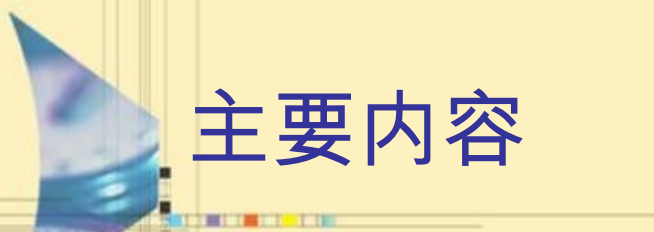


4.1.4 简单问题例算法的分析

- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。
- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。

Those who cannot remember the past are doomed to repeat it.

-----George Santayana,
(1905)



主要内容

- 4.1 引言
- 4.2 动态规划策略的算法设计思想及分析方法
- 4.3 动态规划策略的经典应用（可选讲部分）



主要内容

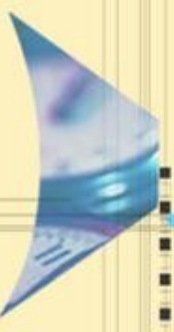
4.2 动态规划策略的算法设计思想及分析方法

4.2.1 动态规划策略适用的问题分析

4.2.2 动态规划算法一般框架及特点分析

4.2.3 动态规划算法设计的关键因素

4.2.4 动态规划和分治策略的异同点



4.2.1 动态规划策略适用的问题分析

- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题。
- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。
- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。



4.2.2 动态规划算法一般框架及特点分析

- 动态规划的基本步骤

- 1) 找出最优解的性质，并刻画其结构特征。
- 2) 递归地定义最优值。
- 3) 以自底向上的方式计算出最优值。
- 4) 根据计算最优值时得到的信息，构造最优解。



4.2.3 动态规划算法设计的关键因素 - 最优子结构

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。
- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。最优子结构是问题能用动态规划算法求解的前提。

同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）



4.2.3 动态规划算法设计的关键因素 - 重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。



4.2.4 动态规划和分治策略的异同点

- 共同点：

1. 动态规划与分治方法相似，通过合并多个子问题的解来解决整体问题。

- 区别：

1. **分治法**是把大问题分解成一些**相互独立的**子问题，**递归**的求解这些子问题然后将他们**合并**来得到整个问题的解
2. **动态规划**是通过组合子问题的解来解决整个大问题。各个子问题**不是独立的**，也就是各个子问题包含公共子问题。它可以避免遇到的子问题的重复求解。





主要内容

- 4.1 引言
- 4.2 动态规划策略的算法设计思想及分析方法
- 4.3 动态规划策略的经典应用（可选讲部分）



主要内容

4.3 动态规划策略的经典应用（可选讲部分）

4.3.1 所有点对最短路径问题

4.3.2 0/1 背包问题

4.3.3 多段图问题

4.3.4 最优二分检索树问题

4.3.5 货郎担问题

4.3.6 可靠性设计问题

4.3.7 流水线调度问题



4.3.1 所有点对最短路径问题

? 问题描述：

- 设 $G = (V, E)$ 是一个有向图，其中的每条边 (i, j) 有一个非负的长度 $l[i, j]$ ，如果从顶点 i 到顶点 j 没有边，则 $l[i, j] = \infty$ 。
- 问题是要找出从每个顶点到其他所有顶点的距离，这里，从顶点 x 到顶点 y 的距离是指从 x 到 y 的最短路径的长度。

4.3.1 所有点对最短路径问题

- 如果有一个矩阵 $D=[d(ij)]$ ，其中 $d(ij)>0$ 表示 i 城市到 j 城市的距离。若 i 与 j 之间无路可通，那么 $d(ij)$ 就是无穷大。又有 $d(ii)=0$ 。编写一个程序，通过这个距离矩阵 D ，把任意两个城市之间的最短与其行径的路径找出来。
- 我们可以将问题分解，如何找出最短路径呢，这里还是用到动态规划的知识：
 - 对于任何一个城市而言， i 到 j 的最短距离不外乎存在经过 i 与 j 之间的 k 和不经过 k 两种可能。
 - 所以可以令 $k=1, 2, 3, \dots, n$ (n 是城市的数目)，再检查 $d(ij)$ 与 $d(ik)+d(kj)$ 的值； $d(ik)$ 与 $d(kj)$ 分别是目前为止所知道的 i 到 k 与 k 到 j 的最短距离，因此 $d(ik)+d(kj)$ 就是 i 到 j 经过 k 的最短距离。每当一个 k 查完了， $d(ij)$ 就是目前的 i 到 j 的最短距离。
 - 重复这一过程，最后当递归查完所有的 k 时， $d(ij)$ 里面存放的就是 i 到 j 之间的最短距离了。

4.3.1 所有点对最短路径问题—递推式

- 设 i 和 j 是 V 中两个不同的顶点, $d_{i,j}^k$ 是从 i 到 j , 并且不经过 $\{k+1, k+2, \dots, n\}$ 中任何顶点的最短路径的长度。
- $d_{i,j}^0 = l[i, j]$, $d_{i,j}^1$ 是从 i 到 j , 除了可能经过顶点 1 以外, 不经过任何其他顶点的最短路径。
 $d_{i,j}^2$ 是从 i 到 j , 除了可能经过顶点 1、顶点 2 或同时经过它们以外, 不经过任何其他顶点的最短路径, 等等。这样由定义可知, $d_{i,j}^n$ 是从 i 到 j 的最短路径长度, 也就是从 i 到 j 的距离。给出这个定义, 可以递归地计算 $d_{i,j}^k$

$$d_{i,j}^k = \begin{cases} l[i, j] & \text{if } k = 0 \\ \min \{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\} & \text{if } 1 \leq k \leq n \end{cases}$$

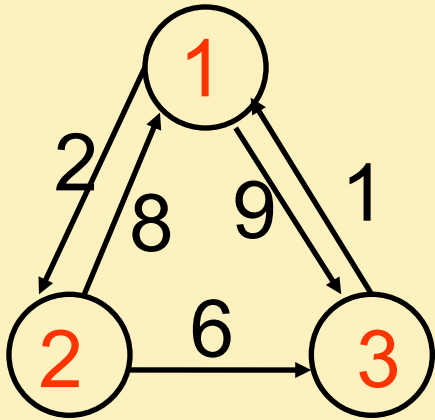
4.3.1 所有点对最短路径问题—Floyd 算法

- 下面是 **Floyd** 设计出的算法，用自底向上地解上面的递推式的方法来处理。
- 它用 $n+1$ 个 $n \times n$ 维矩阵 $D_0, D_1, \dots, D_n$ 来计算最短约束路径的长度。
- 开始时，如果 $i \neq j$ 并且 (i,j) 是 G 中的边，则置 $D_0[i,i] = 0$ ， $D_0[i,j] = l[i,j]$ ；否则置 $D_0[i,j] = \infty$ 。
- 然后做 n 次迭代，使在第 k 次迭代后， $D_k[i,j]$ 含有从顶点 i 到顶点 j ，且不经过编号大于 k 的任何顶点的最短路径的长度。
- 这样在第 k 次迭代中，可以用公式计算 $D_k[i,j]$ ：

$$D_k[i,j] = \min\{D_{k-1}[i,j], D_{k-1}[i,k] + D_{k-1}[k,j]\}$$

4.3.1 所有点对最短路径问题—例子

考虑图 7.4 所示的有向图



矩阵

D_0, D_1, D_2

and D_3 是：

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & \infty & 0 \end{bmatrix}$$

$$D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

4.3.1 所有点对最短路径问题—伪代码

算法 7.3 FLOYD

输入： $n \times n$ 维矩阵 $l[1..n, 1..n]$ ，对于有向图 $G=(\{1, 2, \dots, n\}, E)$ 中的边 (i, j) 的长度是 $l[i, j]$ 。

输出：矩阵 D 使得 $D[i, j]$ = i 到 j 的距离。

```
1.  $D \leftarrow l$            { 将输入矩阵  $l$  复制到  $D$  }
2. for  $k \leftarrow 1$  to  $n$ 
3.   for  $i \leftarrow 1$  to  $n$ 
4.     for  $j \leftarrow 1$  to  $n$ 
5.        $D[i, j] = \min\{D[i, j], D[i, k] + D[k, j]\}$ 
6.     end for
7.   end for
8. end for
```

- 显然，算法的运行时间是 $\Theta(n^3)$ ，它的空间复杂性是 $\Theta(n^2)$ 。

4.3.1 所有点对最短路径问题—空间复杂度

- 一个重要的发现是，在第 k 次迭代中第 k 行和第 k 列都是不变的，因此可以仅用 D 矩阵的一个副本来进行计算。仅用一个 $n \times n$ 矩阵执行这个计算的算法由算法 FLOYD 给出。

4.3.1 所有点对最短路径问题—Floyd 与 Dijkstra 算法比较

- Dijkstra 算法，图所有边权值都为非负的；Floyd 算法，不允许所有权值为负的回路，
- Dijkstra 只可以求出任意点到达源点的最短距离；Floyd 可以求出任意两点间的最短距离，
- Dijkstra 算法的思想是贪心，Floyd 算法的思想是动态规划
- Dijkstra 时间复杂度 $O(n^2)$ ，Floyd 算法 $O(n^3)$
- Dijkstra 算法在网络中用得更多，一个一个节点添加，加一个点刷一次路由表。Floyd 算法把所有已经连接的路径都标出来，再通过不等式比较来更改路径。



4.3.2 0/1 背包问题



问题描述：

- 背包问题可以定义如下
 - 设 $U = \{u_1, u_2, \dots, u_n\}$ 是一个准备放入容量为 C 的背包中的 n 项物品的集合。
 - 对于 $1 \leq j \leq n$ ，令 s_j 和 v_j 分别为第 j 项物品的体积和价值，这里， C ， s_j ， v_j 和 j 都是正整数
 - 我们要解决的问题是用 U 中的一些物品来装满背包，这些物品的总体积不超过 C ，然而要使它们的总价值最大。

4.3.2 0/1 背包问题

- 更形式地, 给出有 n 项物品的 U , 我们要找出一个子集合 S
- 使得 $\sum_{u_i \in S} v_i$ 在约束条件 $\sum_{u_i \in S} s_i \leq C$ 下最大。

4.3.2 0/1 背包问题—基本思想

- 设 $V[i,j]$ 用来表示从前 i 项 $\{u_1, \dots, u_i\}$ 中取出来的装入体积为 j 的背包的物品的最大价值。这里， i 的范围是从 0 到 n ， j 的范围是 0 到 C 。
- $V[0,j]$ 对于所有 j 的值是 0，这是由于背包中什么也没有。另一方面， $V[i,0]$ 对于所有 i 的值为 0，因为没有东西可放到为 0 的背包里。

4.3.2 0/1 背包问题—基本思想

- **背包问题特点**是：每种物品仅有一件，可以选择放或不放。
- **用子问题定义状态**：即 $V[i,j]$ 表示前 i 件物品恰放入一个容量为 j 的背包可以获得的**最大价值**。则其**状态转移方程**便是：
$$V[i,j] = \max \{V[i-1,j], V[i-1, j-s_i] + v_i\}。$$
- 针对“将前 i 件物品放入容量为 v 的背包中”这个子问题，若只考虑第 i 件物品的策略（放或不放），那么就可以转化为一个只牵扯前 $i-1$ 件物品的问题。
 - 如果不放第 i 件物品，那么问题就转化为“前 $i-1$ 件物品放入容量为 j 的背包中”，价值为 $V[i-1,j]$ ；
 - 如果放第 i 件物品，那么问题就转化为“前 $i-1$ 件物品放入剩下的容量为 $j-s_i$ 的背包中”，此时能获得的最大价值就是 $V[i-1, j-s_i]$ 再加上通过放入第 i 件物品获得的价值 v_i 。

4.3.2 0/1 背包问题—观察结论

$V[i,j]$ 是下面两个量的最大值：

1. $V[i-1,j]$: 仅用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 j 的背包所得到的价值最大值。
2. $V[i-1,j-s_i]+v_i$: 用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 $j-s_i$ 的背包所得到的价值最大值加上物品 u_i 的价值。这仅应用于如果 $j \geq s_i$ 以及它等于把物品 u_i 加到背包上的情况。

$$V[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ V[i-1,j] & \text{if } j < s_i \\ \max\{V[i-1,j], V[i-1,j-s_i]+v_i\} & \text{if } i > 0 \text{ and } j \geq s_i \end{cases}$$

4.3.2 0/1 背包问题—伪代码

算法 7.4 KNAPSACK

输入：物品集合 $U=\{u_1, u_2, \dots, u_n\}$ ，体积分别为 $s_1, s_2, \dots, s_n$ ，价值分别为 $v_1, v_2, \dots, v_n$ ，容量为 C 的背包。

输出： $\sum_{u_i \in S} v_i$ 的最大总价值，且满足 $\sum_{u_i \in S} s_i \leq C$ ，其中 $S \subseteq U$ 。

```
1. for i ← 0 to n
2.      $V[i, 0] \leftarrow 0$ 
3. end for
4. for j ← 0 to n
5.      $V[0, j] \leftarrow 0$ 
6. end for
7. for i ← 1 to n
8.     for j ← 1 to C
9.          $V[i, j] \leftarrow V[i-1, j]$ 
10.        if  $s_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-s_i] + v_i\}$ 
11.     end for
12. end for
13. return  $V[n, C]$ 
```

4.3.2 0/1 背包问题—例子

- 容量 : 9; 4 种体积的物品 : 2, 3, 4, 5; 价值分别为 : 3, 4, 5, 7.

The maximum value is 12 and there are two optimal solutions:
1,2,3 and 3,4

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12
4	0	0	3	4	5	7	8	10	11	12

4.3.2 0/1 背包问题—算法分析

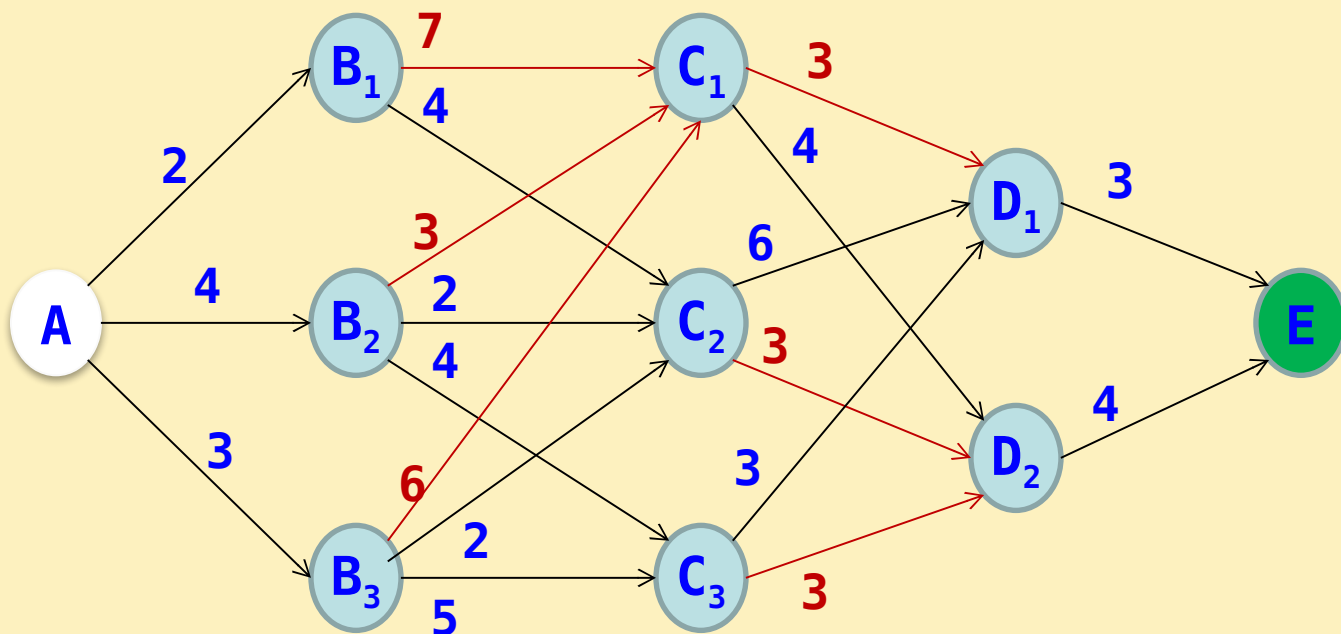
- 很明显，由于计算表的每一项需要 $\Theta(1)$ 时间，算法的时间复杂度恰好是表的大小 $\Theta(nC)$ 。
- 由于计算当前行只需要上一次计算的行，因此对算法稍作修改可以使其只需要 $\Theta(C)$ 的空间。
- 定理
 - 背包问题的最优解能够在 $\Theta(nC)$ 的时间内和 $\Theta(C)$ 的空间内得到。

4.3.3 多段图问题

设图 $G=(V, E)$ 是一个带权有向连通图，如果把顶点集合 V 划分成 k 个互不相交的子集 V_i ($2 \leq k \leq n$, $1 \leq i \leq k$)，使得 E 中的任何一条边 (u, v) ，必有 $u \in V_i$, $v \in V_{i+m}$ ($1 \leq i < k$, $1 < i+m \leq k$)，则称图 G 为多段图，称 $s \in V_1$ 为源点， $t \in V_k$ 为终点。多段图的最短路径问题是求从源点到终点的最小代价路径。

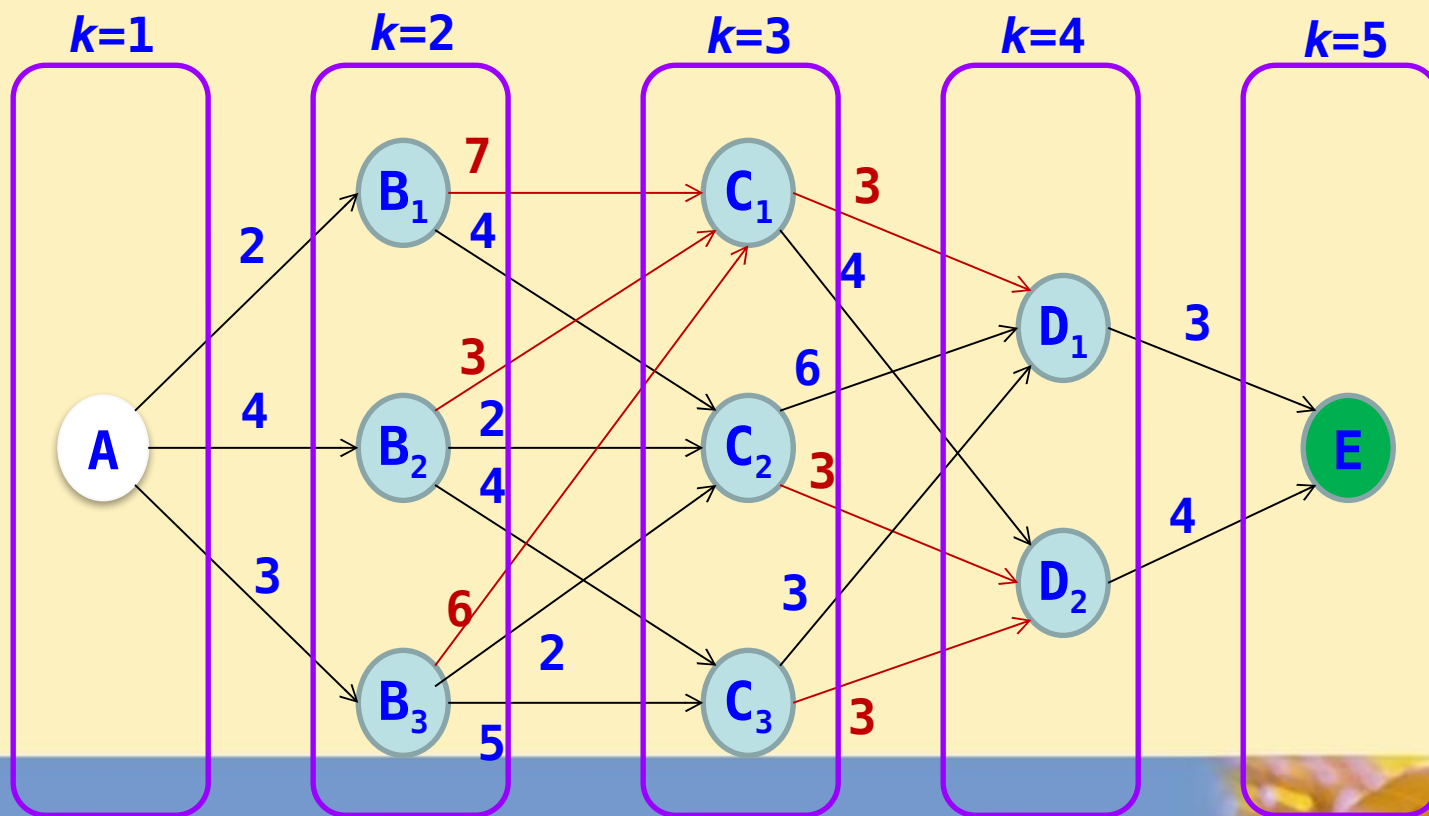
4.3.3 多段图问题

多段图问题示例：在 A 处有一水库，现需要从 A 点铺设一条管道到 E 点，边上的数字表示与其相连的两个地点之间所需修建的管道长度，用 c 数组表示，例如 $c(A, B_1)=2$ 。现要找出一条从 A 到 E 的修建线路，使得所需修建的管道长度最短。

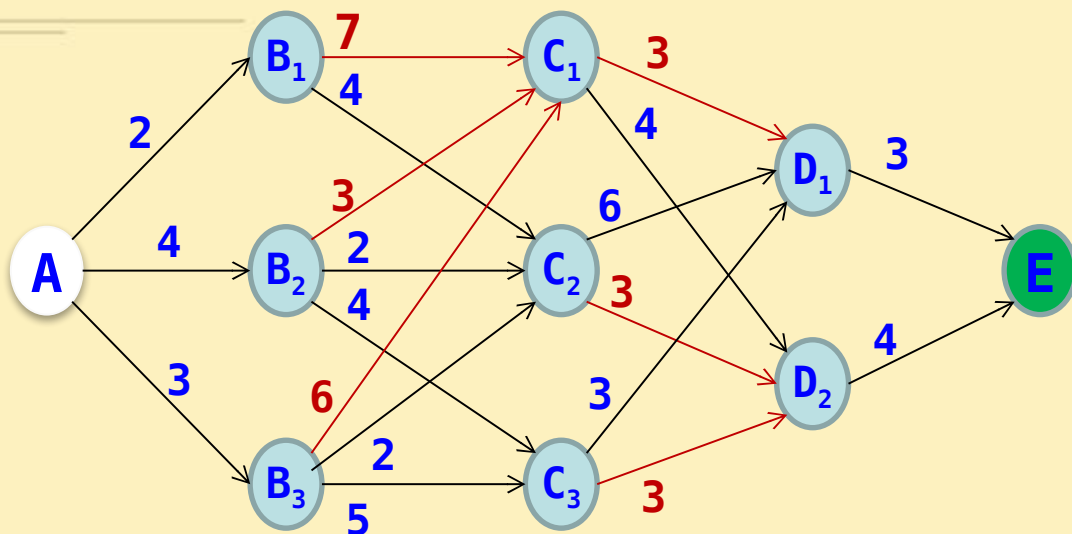


4.3.3 多段图问题

在从 $A \sim E$ 的过程中，依据按位置所做的决策的次数及所做决策的先后次序，将问题分为 5 个阶段，阶段变量用于表示各阶段，这里阶段变量 k 为 1 ~ 5，图中第 5 阶段是虚拟的一个边界阶段。



4.3.3 多段图问题



设最优指标函数 $f(s)$ 表示状态 s 到终点 E 的最短路径长度, 用 k 表示阶段, 则对应的状态转移方程如下:

$$f_5(E) = 0$$

$$f_k(s_k) = \min_{x_k \in D_k(s_k)} \{c(s_k, x_k) + f_{k+1}(s_{k+1})\}$$

符号示例

$$S_2 = \{B_1, B_2, B_3\}$$

s_2 取 S_2 的元素

$$D_2(B_1) = \{C_1, C_2\}$$

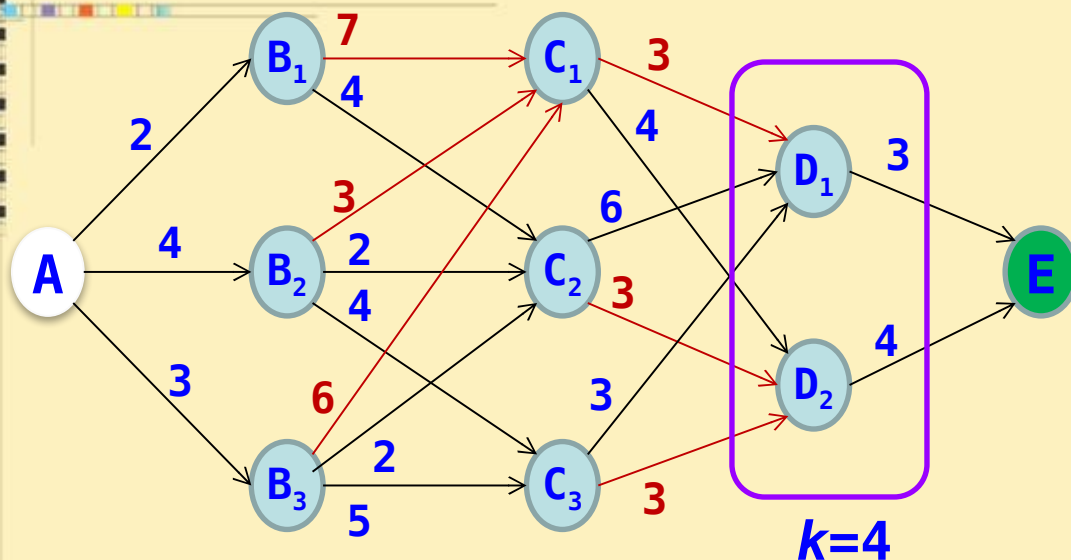
x_2 取 $D_2(B_1)$ 的元素

4.3.3 多段图问题

- 逆序解法
- 顺序解法



4.3.3 多段图问题 - 逆序解法



$f(s)$ 表示状态 s 到
终点 E 的最短路径
长度

求解 $E \rightarrow A$ 的过程（用 $next$ 表示路径上一个顶点的后继顶点）：

① 第 5 阶段

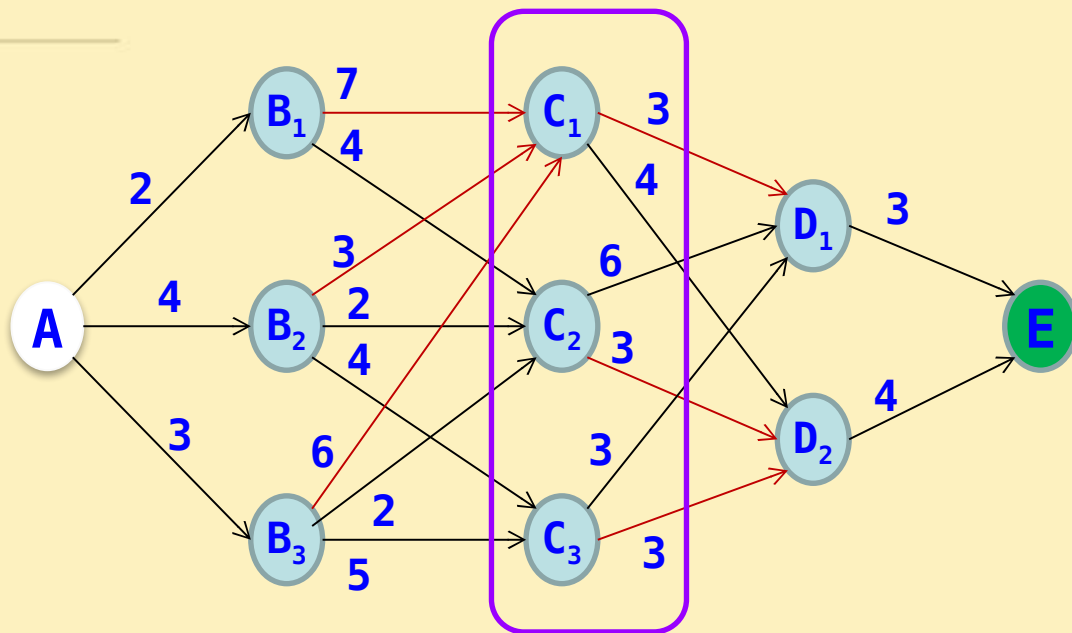
$$f(E)=0$$

② 第 4 阶段

$$f(D_1)=\text{MIN}(c(D_1, E)+f(E))=3, \quad \text{next}(D_1)=E$$

$$f(D_2)=\text{MIN}(c(D_2, E)+f(E))=4, \quad \text{next}(D_2)=E$$

4.3.3 多段图问题 - 逆序解法



③ 第 3 阶段

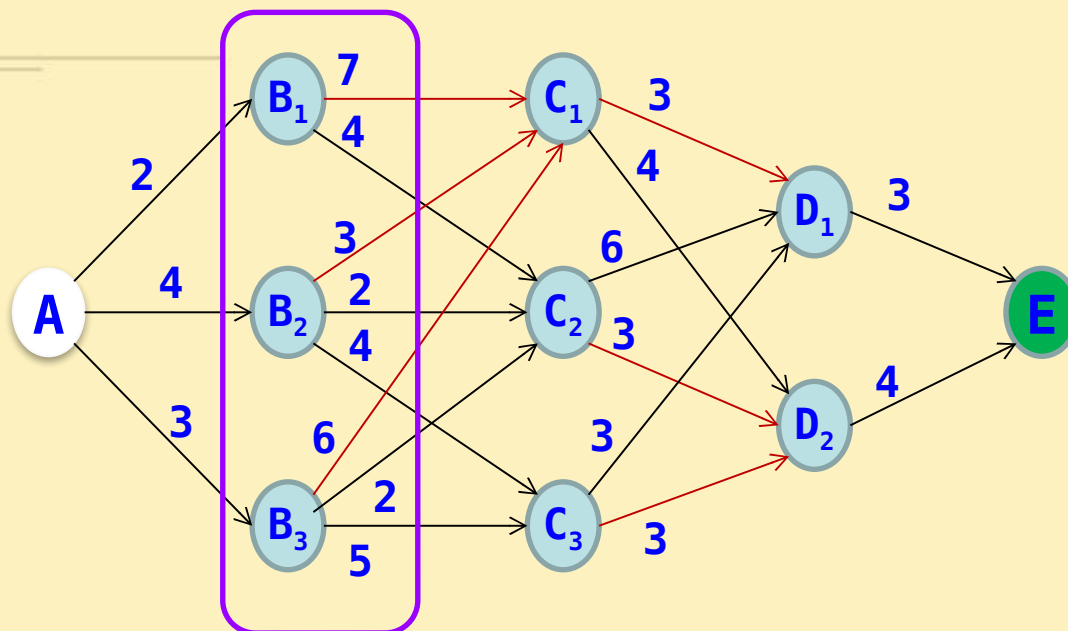
$k=3$

$$f(C_1) = \min \begin{cases} c(C_1, D_1) + f(D_1) = 6 \\ c(C_1, D_2) + f(D_2) = 8 \end{cases} = 6, \text{ next}(C_1)=D_1$$

$$f(C_2) = \min \begin{cases} c(C_2, D_1) + f(D_1) = 9 \\ c(C_2, D_2) + f(D_2) = 7 \end{cases} = 7, \text{ next}(C_2)=D_2$$

$$f(C_3) = \min \begin{cases} c(C_3, D_1) + f(D_1) = 6 \\ c(C_3, D_2) + f(D_2) = 7 \end{cases} = 6, \text{ next}(C_3)=D_1$$

4.3.3 多段图问题 - 逆序解法



④ 第 2 阶段

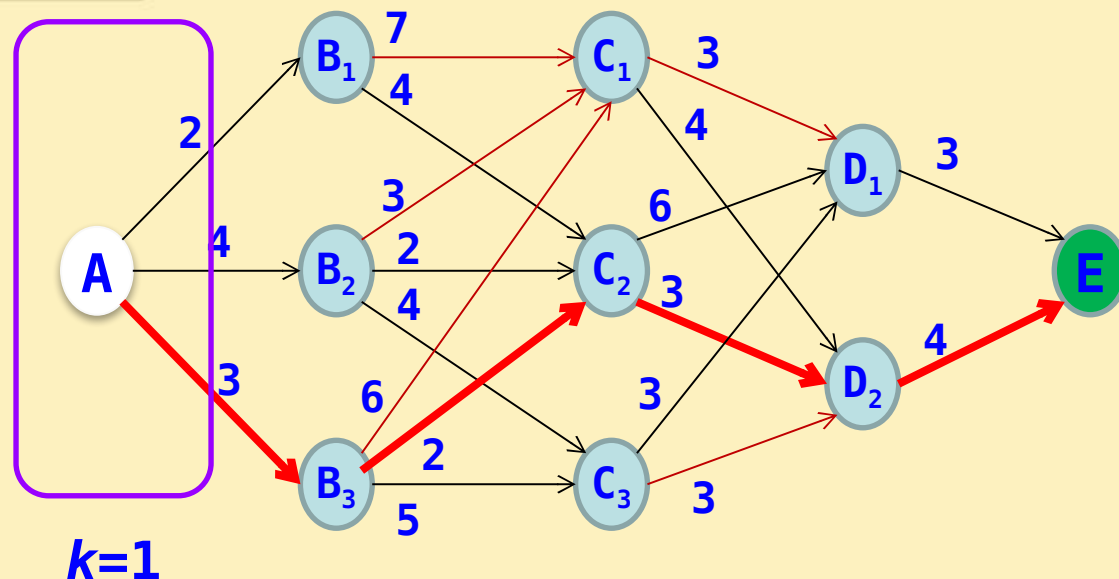
$k=2$

$$f(B_1) = \min \begin{pmatrix} c(B_1, C_1) + f(C_1) = 13 \\ c(B_1, C_2) + f(C_2) = 11 \\ c(B_1, C_3) + f(C_3) = \infty \end{pmatrix} = 11, \text{ next}(B_1) = C_2$$

$$f(B_2) = \min \begin{pmatrix} c(B_2, C_1) + f(C_1) = 9 \\ c(B_2, C_2) + f(C_2) = 9 \\ c(B_2, C_3) + f(C_3) = 10 \end{pmatrix} = 9, \text{ next}(B_2) = C_1$$

$$f(B_3) = \min \begin{pmatrix} c(B_3, C_1) + f(C_1) = 12 \\ c(B_3, C_2) + f(C_2) = 9 \\ c(B_3, C_3) + f(C_3) = 11 \end{pmatrix} = 9, \text{ next}(B_3) = C_2$$

4.3.3 多段图问题 - 逆序解法



⑤ 第 1 阶段

$$f(A) = \min \begin{pmatrix} c(A, B_1) + f(B_1) = 13 \\ c(A, B_2) + f(B_2) = 13 \\ c(A, B_3) + f(B_3) = 12 \end{pmatrix} = 12, \text{ next}(A) = B_3$$

由 $f(A)=12$ 求出的最短路径长度为 12

由 $\text{next}(A)=B_3$, $\text{next}(B_3)=C_2$, $\text{next}(C_2)=D_2$, $\text{next}(D_2)=E$, 推出最短路径为 $A \rightarrow B_3 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ 。

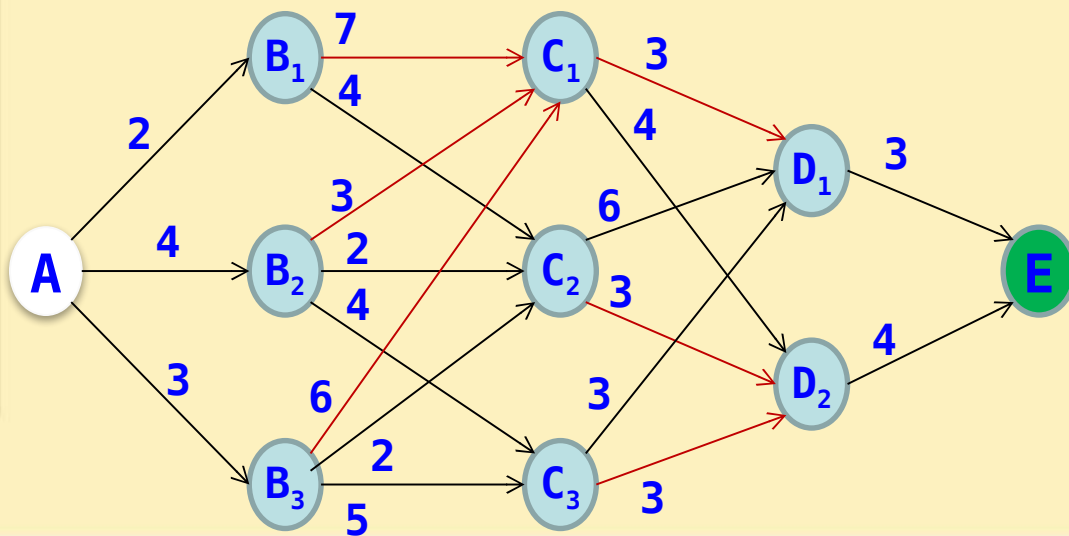
4.3.3 多段图问题 - 顺序解法

$A \rightarrow E$ 对应的的状态转移方程如下:

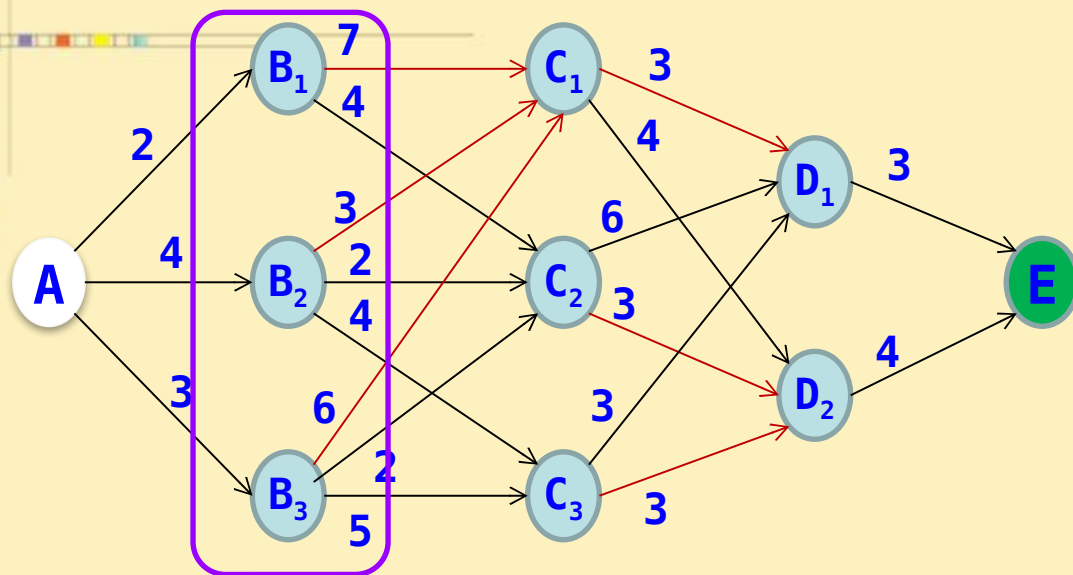
$$f(A) = 0$$

$$f(s) = \min_{\text{存在 } \langle t, s \rangle \text{ 的有向边}} \{f(t) + c(t, s)\}$$

$f(s)$ 表示初始状态 A
到状态 s 的最短路径长
度



4.3.3 多段图问题 - 顺序解法



$f(s)$ 表示初始状态 A 到状态 s 的最短路径长度

用 pre 表示路径上一个顶点的前驱顶点，其求解 $A \rightarrow E$ 的过程：

① 第 1 阶段

$$f(A)=0$$

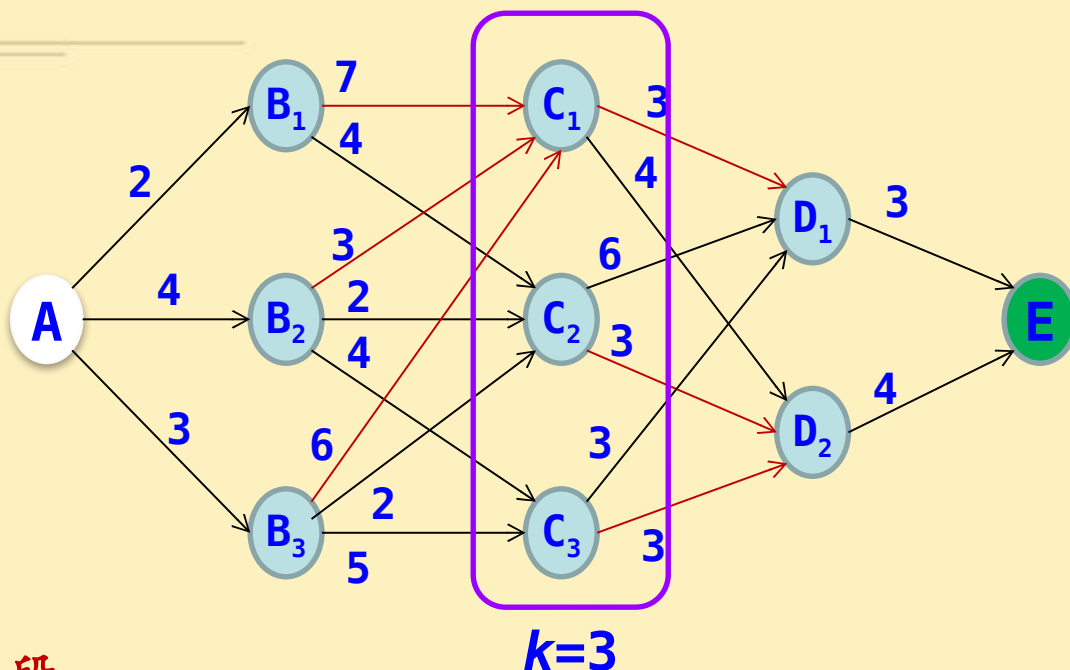
② 第 2 阶段

$$f(B_1)=\min(f(A)+c(A, B_1))=2, \quad pre(B_1)=A$$

$$f(B_2)=\min(f(A)+c(A, B_2))=4, \quad pre(B_2)=A$$

$$f(B_3)=\min(f(A)+c(A, B_3))=3, \quad pre(B_3)=A$$

4.3.3 多段图问题 - 顺序解法



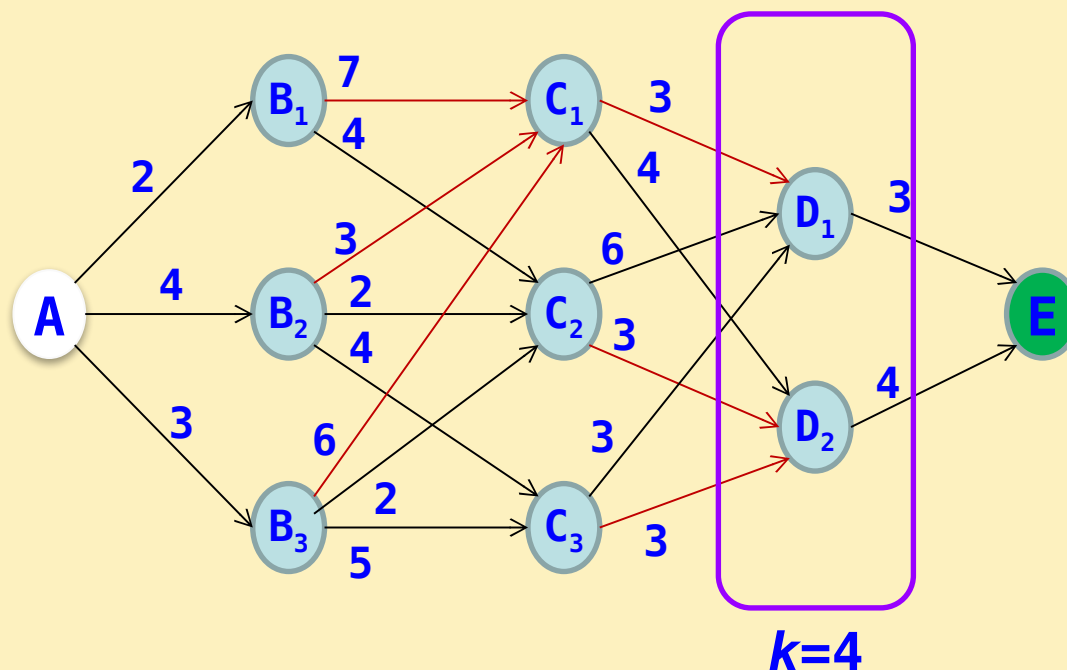
③ 第 3 阶段

$$f(C_1) = \min \begin{pmatrix} f(B_1) + c(B_1, C_1) = 9 \\ f(B_2) + c(B_2, C_1) = 7 \\ f(B_3) + c(B_3, C_1) = 9 \end{pmatrix} = 7, \text{ pre}(C_1)=B_2$$

$$f(C_2) = \min \begin{pmatrix} f(B_1) + c(B_1, C_2) = 6 \\ f(B_2) + c(B_2, C_2) = 6 \\ f(B_3) + c(B_3, C_2) = 5 \end{pmatrix} = 5, \text{ pre}(C_2)=B_3$$

$$f(C_3) = \min \begin{pmatrix} f(B_1) + c(B_1, C_3) = \infty \\ f(B_2) + c(B_2, C_3) = 8 \\ f(B_3) + c(B_3, C_3) = 8 \end{pmatrix} = 8, \text{ pre}(C_3)=B_2$$

4.3.3 多段图问题 - 顺序解法



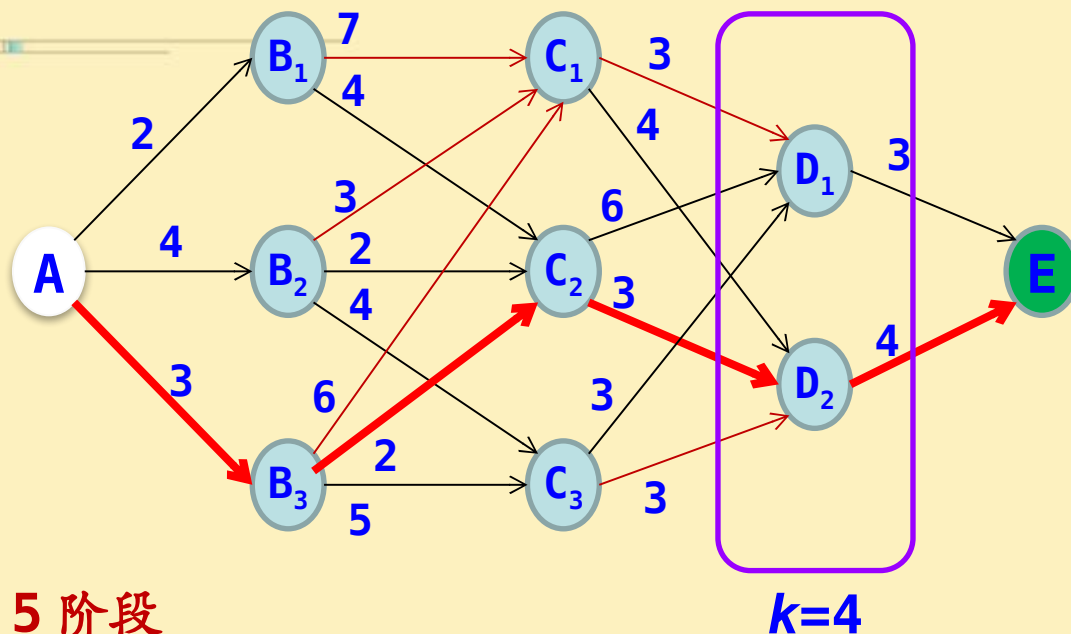
④ 第 4 阶段

$$f(D_1) = \min \begin{pmatrix} f(C_1) + c(C_1, D_1) = 10 \\ f(C_2) + c(C_2, D_1) = 11 \\ f(C_3) + c(C_3, D_1) = 11 \end{pmatrix} = 10, \text{ pre}(D_1) = C_1$$

$$f(D_2) = \min \begin{pmatrix} f(C_1) + c(C_1, D_2) = 11 \\ f(C_2) + c(C_2, D_2) = 8 \\ f(C_3) + c(C_3, D_2) = 11 \end{pmatrix} = 8, \text{ pre}(D_2) = C_2$$



4.3.3 多段图问题 - 顺序解法



⑤ 第 5 阶段

$$f(E) = \min \begin{pmatrix} f(D_1) + c(D_1, E) = 13 \\ f(D_2) + c(D_2, E) = 12 \end{pmatrix} = 12, \text{ pre}(E) = D_2$$

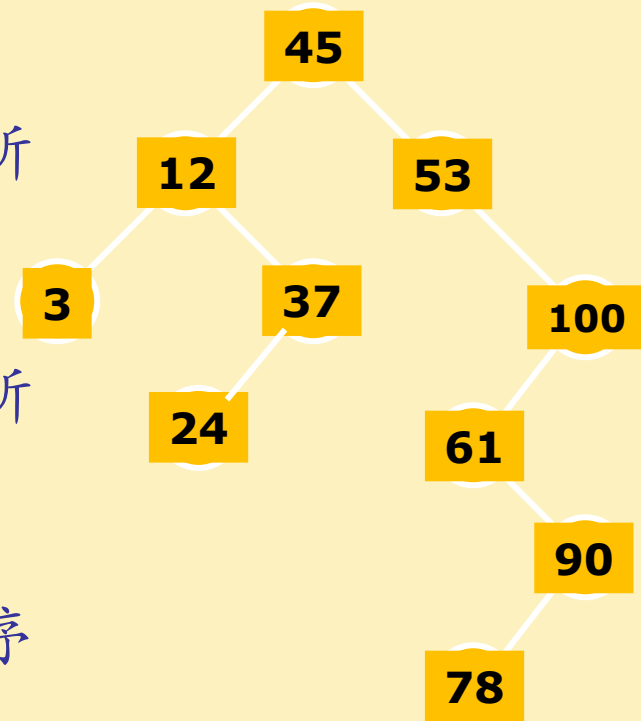
由 $f(E)=12$ 求出的最短路径长度为 12

由 $\text{pre}(E)=D_2$, $\text{pre}(D_2)=C_2$, $\text{pre}(C_2)=B_3$, $\text{pre}(B_3)=A$, 推出最

短路径为 $A \rightarrow B_3 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ 。

4.3.4 最优二分检索树问题

- 什么是二叉搜索树?
 - 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值；
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值；
 - 它的左、右子树也分别为二叉排序树
- 在随机的情况下，二叉查找树的平均查找长度和 $\log n$ 是等数量级的。



4.3.4 最优二分检索树问题

- 具体地，设 $S=\{x_1, x_2, \dots, x_n\}$ 是一个有序集，且 $x_1 < x_2 < \dots < x_n$ 。表示有序集 S 的二叉搜索树利用二叉树的结点来存储有序集中的元素。它具有下述性质：存储于每个结点中的元素 x 大于其左子树中任一结点所存储的元素，小于其右子树中任一结点所存储的元素。二叉搜索树的叶结点是形如 (x_i, x_{i+1}) 的开区间。在表示 S 的二叉搜索树中搜索一个元素 x ，返回的结果有两种情形：
 - 在二叉搜索树的内结点中找到 $x=x_i$ 。
 - 在二叉搜索树的叶结点中确定 $x \in (x_i, x_{i+1})$ 。

4.3.4 最优二分检索树问题

- 设第一种情形中找到元素 $x=x_i$ 的概率为 b_i ；在第二种情形中确定 $x \in (x_i, x_{i+1})$ 的概率为 a_i 。并约定 $x_0 = -\infty$, $x_{n+1} = +\infty$ 。显然有

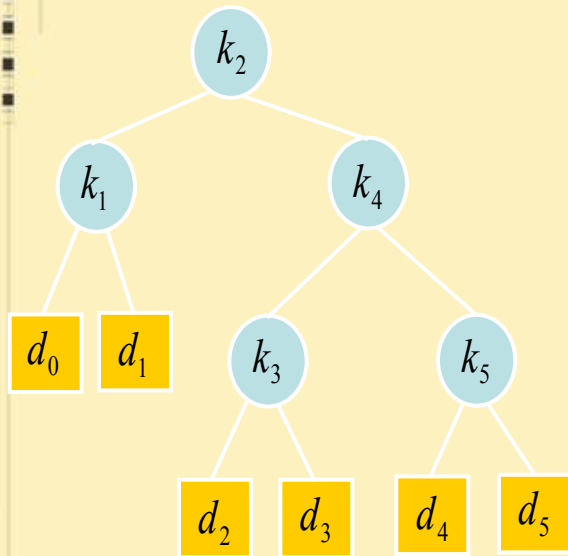
$$a_i \geq 0, 0 \leq i \leq n; \quad b_j \geq 0, 0 \leq j \leq n; \quad \sum_{i=0}^n a_i + \sum_{j=1}^n b_j = 1$$

- $(a_0, b_1, a_1, \dots, b_n, a_n)$ 称为集合 S 的存取概率分布。
- 在表示 S 的二叉搜索树 T 中，设存储元素 x_i 的结点深度为 c_i ；存储叶结点 (x_j, x_{j+1}) 的结点深度为 d_j ，则

$$p = \sum_{i=1}^n b_i(1 + c_i) + \sum_{j=0}^n a_j d_j$$

- 表示在二叉搜索树 T 中作一次搜索所需要的平均比较次数。 P 又称为二叉搜索树 T 的平均路长。在一般情况下，不同的二叉树的平均路长时不相同的。

4.3.4 最优二分检索树问题—期望耗费



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

4.3.4 最优二分检索树问题

- 最优二叉搜索树问题是对于有序集 S 及其存取概率分布 $(a_0, b_1, a_1, \dots, b_n, a_n)$ ，在所有表示有序集 S 的二叉搜索树中找出一棵具有最小平均路长的二叉搜索树。
- 有 n 个节点的二叉树的个数为： $\Omega(4^n/n^{3/2})$ 。穷举搜索法的时间复杂度为指数级。

4.3.4 最优二分检索树问题—最优子结构

- 二叉搜索树 T 的一棵含有结点 $x_i, \dots, x_j$ 和叶结点 $(x_{i-1}, x_i), \dots, (x_j, x_{j+1})$ 的子树可以看作是有序集 $\{x_i, \dots, x_j\}$ 关于全集合 $\{x_{i-1}, x_{j+1}\}$ 的一棵二叉搜索树，其存取概率为下面的条件概率

$$\begin{aligned} \overline{b_k} &= b_k / w_{ij}, \quad i \leq k \leq j; \overline{a_h} = a_h / w_{ij}, \quad i-1 \leq h \leq j; \\ (p) \quad &\{\overline{a_{i-1}}, \overline{b_i}, \dots, \overline{b_j}, \overline{a_j}\} \end{aligned}$$

- 其中， $w_{ij} = a_{i-1} + b_i + \dots + b_j + a_j$ 。
- 设 T_{ij} 是有序集合 $\{x_i, \dots, x_j\}$ 关于存取概率 P 的一棵最优二叉搜索树，其平均路长为 p_{ij} 。 T_{ij} 的根结点存储元素 x_m ，其左右子树 T_l 和 T_r 的平均路长分别为 p_l 和 p_r 。由于 T_l 和 T_r 中结点深度是他们在 T_{ij} 中的结点深度减 1，故我们有 $w_{i,j} p_{i,j} = w_{i,j} + w_{i,m-1} p_l + w_{m+1,j} p_r$

- 由于 T_l 是关于集合 $\{x_i, \dots, x_{m-1}\}$ 的一棵二叉搜索树，故 $p_l \geq p_{i,m-1}$ 。若 $p_l > p_{i,m-1}$ ，则用 $T_{i,m-1}$ 替换 T_l 可得到平均路长比 T_{ij} 更小的二叉搜索树。这与 T_{ij} 是最优二叉搜索树矛盾。故 T_{ij} 是一个最优二叉搜索树。同理可证也是一棵最优二叉搜索树。
- 因此最优二叉搜索树具有最优子结构性质。

4.3.4 最优二分检索树问题—最优子结构

- 最优二叉搜索树 T_{ij} 的平均路长为 p_{ij} ，则所求的最优值为 $p_{1,n}$ 。由最优二叉搜索树问题的最优子结构性质可建立计算 p_{ij} 的递归式如下。

$$w_{i,j}p_{i,j} = w_{i,j} + \min_{i \leq k \leq j} \{w_{i,k-1}p_{i,k-1} + w_{k+1,j}p_{k+1,j}\} \quad p_{i,i-1} = 0, 1 \leq i \leq n$$

- 记 $w_{i,j}p_{i,j}$ 为 $m(i,j)$ ，则 $m(1,n)=w_{1,n}p_{1,n}=p_{1,n}$ 为所求的最优值。计算 $m(i,j)$ 的递归式为

$$m(i,j) = w_{i,j} + \min_{i \leq k \leq j} \{m(i,k-1) + m(k+1,j)\}, \quad i \leq j$$

$$m(i,i-1) = 0, \quad 1 \leq i \leq n$$

- 注意到，

$$\min_{i \leq k \leq j} \{m(i,k-1) + m(k+1,j)\} = \min_{s[i][j-1] \leq k \leq s[i+1][j]} \{m(i,k-1) + m(k+1,j)\}$$

- 可以得到 $O(n^2)$ 的算法



4.3.5 货郎担问题

- 货郎担问题也叫旅行商题 (traveling salesman problem) ,
- 其一般提法为 : 有 n 个城市, 用 $1, 2, \dots, n$ 表示, 城 i, j 之间的距离为 d_{ij} , 有一个货郎从城 1 出发到其他城市一次且仅一次, 最后回到城市 1, 怎样选择行走路线使总路程最短?

4.3.5 货郎担问题

- 阶段变量 k : 按所经过的城市个数划分阶段 k ,
 $k=1, 2, \dots, n-1$.
- 状态变量 S_k : 设第 k 阶段到达城市 i 时途中所经过的 k 个城集合为 S , 则 $S_k = (S, i)$ 其中
- $S \subset \{2, 3, \dots, i-1, i+1, \dots, n\} \quad |S|=k$
- 决策变量 x_k : 第 k 阶段到达城市 i 的最短路线上邻接 i 的
- 前一个城市 j 。
- $j=2, 3, \dots, n$.
- 例如 $x_3(\{2, 3, 4\}, 5)=3$,
- 表示推销商从城 1 出发途径城市 $\{2, 3, 4\}$ 到达城市 5 时,
- 须先途经城市 $\{2, 4\}$ 到达城市 3 , 再去城市 5.

4.3.5 货郎担问题

- 阶段指标函数 d_{ji} : 设从城市 1 出发, 第 $k-1$ 阶段到达城市 j , 则城市与下一阶段 (第 k 阶段) 的目的地城市 i 之间的距离为 d_{ji}
- 最优指标函数 $f_k(S,i)$: 从城市 1 出发, 经过 S 中 k 个城市, 到达城市 i 的最短距离。
- 动态规划的递推关系为:

$$\begin{cases} f_k(S,i) = \min_{j \in S} \{f_{k-1}(S \setminus j, j) + d_{ji}\} \\ f_0(\phi, i) = d_{1i}, i = 2, 3, \dots, n, k = 1, 2, \dots, n-1. \end{cases}$$

最后算出 $f_{n-1}(N,1)$, $N=\{2,3, \dots, n\}$, 即为全程的最短距离, 同时可得最优策略, 即最优行走路线。

4.3.5 货郎担问题

- 例：已知 4 个城市间距离如表 1，求从城市 1 出发，经其与其它城市一次且仅一次最优回到城市 1 的最短路与距离。

城市 \ 城市	1	2	3	4
1	0	6	7	9
2	8	0	9	7
3	5	8	0	8
4	6	5	5	0

解：由边界条件知：

$$f_0(\phi, 1) = d_{11} = 0 \quad f_0(\phi, 2) = d_{12} = 6 \quad f_0(\phi, 3) = d_{13} = 7$$

$$f_0(\phi, 4) = d_{14} = 9$$

4.3.5 货郎担问题

- 当 $k=1$ 时，从城市 1 出发，经过 1 个城市到达城市 i 的最短距离为

$$\begin{aligned} f_1(S,2) &= \min\{ f_0(\phi,3)+d_{32}, f_0(\phi,4)+d_{42} \} \\ &= \min\{ 7+8, 9+5 \} = 14, \quad x_2^*(\{4\},2) = 4 \end{aligned}$$

即从城市 1 出发，途经 1 个城市去城 2，应先到 4，再到 2.

$$\begin{aligned} f_1(S,3) &= \min\{ f_0(\phi,2)+d_{23}, f_0(\phi,4)+d_{43} \} \\ &= \min\{ 6+9, 9+5 \} = 14, \\ x_2^*(\{4\},3) &= 4 \end{aligned}$$

即从城市 1 出发，途经 1 个城市去城 3，应先到 4，再到 3a

4.3.5 货郎担问题

$$\begin{aligned} f_1(S,4) &= \min\{ f_0(\phi,2) + d_{24}, f_0(\phi,3) + d_{34} \} \\ &= \min\{ 6 + 7, 7 + 8 \} = 13, \\ x_2^*(\{2\},4) &= 2 \end{aligned}$$

- 即从城市 1 出发，途经 1 个城市去城 4，应先到 2，再到 4。
- 当 $k=2$ 时，从城市 1 出发，途经 2 个城市到达城市 i 的最短距离为 $f_1(\{3\},4)=15$ $f_1(\{4\},3)=14$

$$\begin{aligned} f_2(\{3,4\},2) &= \min\{ f_1(\{3\},4) + d_{42}, f_1(\{4\},3) + d_{32} \} \\ &= \min\{ 15 + 5, 14 + 8 \} = 20 \\ x_2^*(\{3,4\},2) &= 4 \end{aligned}$$

4.3.5 货郎担问题

$$f_1(\{2\},4)=13 \quad f_1(\{4\},2)=14$$

$$\begin{aligned} f_2(\{2,4\},3) &= \min\{ f_1(\{2\},4) + d_{43}, f_1(\{4\},2) + d_{23} \} \\ &= \min\{ 13 + 5, 14 + 9 \} = 18 \end{aligned}$$

$$x_2^*(\{2,4\},3) = 4$$

$$f_1(\{2\},3)=15 \quad f_1(\{3\},2)=15$$

$$\begin{aligned} f_2(\{2,3\},4) &= \min\{ f_1(\{2\},3) + d_{34}, f_1(\{3\},2) + d_{24} \} \\ &= \min\{ 15 + 8, 15 + 7 \} = 22 \end{aligned}$$

$$x_2^*(\{2,3\},4) = 2$$



4.3.5 货郎担问题

- 当 $k=3$ 时，从城市 1 出发，途经 3 个城市到达城市 1 的最短距离

$$f_2(\{2,3\},4)=22 \quad f_2(\{2,4\},3)=18 \quad f_2(\{3,4\},2)=20$$

$$\begin{aligned} f_3(\{2,3,4\},1) &= \min\{ f_2(\{2,3\},4) + d_{41}, f_2(\{2,4\},3) + d_{31}, f_2(\{3,4\},2) + d_{21} \} \\ &= \min\{ 22 + 6, 18 + 5, 20 + 8 \} = 23 \end{aligned}$$

$$x_3^*(\{2,3,4\},1) = 3 \quad \text{逆推回去} \quad x_2^*(\{2,4\},3) = 4; \quad x_2^*(\{2\},4) = 2$$

- 货郎担的最短路线是 $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ 。行走距离为 23

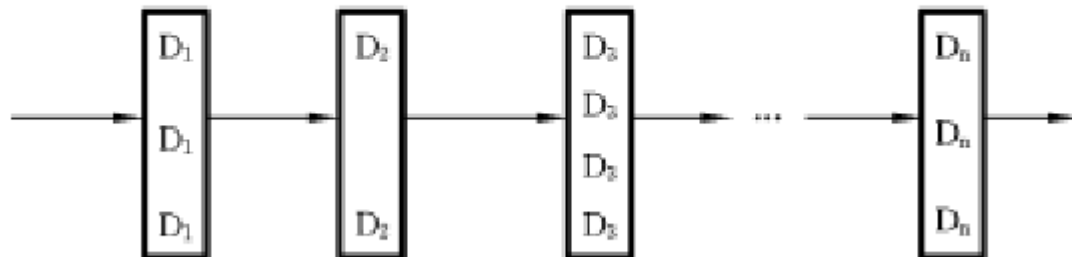


4.3.6 可靠性设计问题

- 可靠性设计：一个系统由 n 级设备串联而成，为了增强可靠性，每级都可能并联了不止一台同样的设备。所以每级用的设备越多系统的可靠性越高。
- 但是设备都是有成本的，假定设备 D_i 的成本是 c_i ，设计该系统允许的投资不超过 c ，那么，该如何设计该系统（即各级采用多少设备）使得这个系统的可靠性最高。试设计一个动态规划算法求解可靠性设计。



图 6.11 以串联方式连接的 n 台设备



每级中以并联方式连接多台设备

4.3.6 可靠性设计问题

- 如果第 i 级的设备 D_i 的台数为 m_i , 那么这 m_i 台设备同时出现故障的概率为 $(1-r_i)^{m_i}$, 从而第 i 级的可靠性就变成 $1-(1-r_i)^{m_i}$ 。例如, 假定 $r_i=0.99$, $m_i=2$, 于是这一级的可靠性就是 0.9999。
- 不过在任何实际系统中, 每一级的可靠性要比 $1-(1-r_i)^{m_i}$ 小一些, 这是由于这些开关线路本身并不是完全可靠的, 而且同一类设备的失误率也不可能是完全独立的 (例如, 由于设计不当所造成的失误)。
- 基于以上分析, 不妨假设第 i 级的可靠性由函数 $\varphi(m_i)$ 给定, $1 \leq i \leq n$ 。由于系统中每种设备至少有一台, 用 u_i 表示最多的台数。

4.3.6 可靠性设计问题

- 例子：设计一个由设备 D1,D2 ,D3 组成的三级系统。每台设备的成本分别为 30 元 ,15 元和 20 元，可靠性分别是 0.9 ,0.8 和 0.5，计划建立该系统的投资不得超过 105 元。假定，若 i 级有 m_i 台设备 D_i 并联，则该级的可靠性 $\varphi(m_i) = 1-(1-r_i)^{m_i}$ 。上述条件可以表示为：
 $c=105$; $c_1=30$, $c_2=15$, $c_3=20$; $r_1=0.9$, $r_2=0.8$, $r_3=0.5$ 。
由此立即可得： $u_1=2$, $u_2=3$, $u_3=3$ 。

4.3.6 可靠性设计问题

- 求解过程
- S_{ij} 表示对第 i 个设备选择 j 个
- S_i 包含了第 i 个设备所有可能的情况。
- $S(i, j)$, S_i 的值都是前面代表可靠性, 后面代表价格。
- 支配: 花了更多的钱, 可靠性反而低, 可以删除。

4.3.6 可靠性设计问题

- $S_{11} = \{ (0.9, 30) \}$ $S_{12} = \{ (0.99, 60) \}$
- 得 $S_1 = \{ (0.9, 30), (0.99, 60) \}$
- 由 $S_{21} = \{ (0.72, 45), (0.792, 75) \}$ $S_{22} = \{ (0.864, 60) \}$ $S_{23} = \{ (0.8928, 75) \}$
- 得 $S_2 = \{ (0.72, 45), (0.864, 60), (0.8928, 75) \}$
- 注意： S_{22} 中已删去了由 $(0.99, 60)$ 所得到的序偶 $(0.9504, 90)$ 。因为这只剩下 15 元，不足以让 $m_3 = 1$ 。
- 说明：归并时由于 $(0.792, 75)$ 受 $(0.864, 60)$ 支配，故舍去。
- 由 $S_{31} = \{ (0.36, 65), (0.432, 80), (0.4464, 95) \}$
- $S_{32} = \{ (0.54, 85), (0.648, 100) \}$
- $S_{33} = \{ (0.63, 105) \}$
- 得 $S_3 = \{ (0.36, 65), (0.432, 80), (0.54, 85), (0.648, 100) \}$
- 最优设计有 0.648 的可靠性，需用资金 100 元。通过对这些 S_i 的回溯，求出 $m_1 = 1, m_2 = 2, m_3 = 2$ 。



4.3.7 流水线调度问题

- n 个作业 $\{1, 2, \dots, n\}$ 要在由 2 台机器 $M1$ 和 $M2$ 组成的流水线上完成加工。每个作业加工的顺序都是先在 $M1$ 上加工，然后在 $M2$ 上加工。 $M1$ 和 $M2$ 加工作业 i 所需的时间分别为 a_i 和 b_i 。
- 流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 $M1$ 上开始加工，到最后最后一个作业在机器 $M2$ 上加工完成所需的时间最少。

4.3.7 流水线调度问题

- 直观上，一个最优调度应使机器 M1 没有空闲时间，且机器 M2 的空闲时间最少。在一般情况下，机器 M2 上会有机器空闲和作业积压 2 种情况。
- 设全部作业的集合为 $N=\{1, 2, \dots, n\}$ 。S 是 N 的作业子集。在一般情况下，机器 M1 开始加工 S 中作业时，机器 M2 还在加工其他作业，要等时间 t 后才可利用。将这种情况下完成 S 中作业所需的最短时间记为 $T(S,t)$ 。流水作业调度问题的最优值为 $T(N,0)$ 。

4.3.7 流水线调度问题

- 设 π 是所给 n 个流水作业的一个最优调度，它所需的加工时间为 $a_{\pi(1)} + T'$ 。其中 T' 是在机器 M_2 的等待时间为 $b_{\pi(1)}$ 时，安排作业 $\pi(2), \dots, \pi(n)$ 所需的时间。记 $S = N - \{\pi(1)\}$ ，则有 $T' = T(S, b_{\pi(1)})$ 。
- 证明：事实上，由 T 的定义知 $T' \geq T(S, b_{\pi(1)})$ 。若 $T' > T(S, b_{\pi(1)})$ ，设 π' 是作业集 S 在机器 M_2 的等待时间为 $b_{\pi(1)}$ 情况下的一个最优调度。则 $\pi(1), \pi'(2), \dots, \pi'(n)$ 是 N 的一个调度，且该调度所需的时间为 $a_{\pi(1)} + T(S, b_{\pi(1)}) < a_{\pi(1)} + T'$ 。这与 π 是 N 的最优调度矛盾。故 $T' \leq T(S, b_{\pi(1)})$ 。从而 $T' = T(S, b_{\pi(1)})$ 。这就证明了流水作业调度问题具有最优子结构的性质。

- 由
$$T(N, 0) = \min_{1 \leq i \leq n} \{a_i + T(N - \{i\}, b_i)\}$$
$$T(S, t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

- 思考：直接利用该递归关系构造动态规划算法。

4.3.7 流水线调度问题—Johnson 不等式

- 对递归式的深入分析表明，算法可进一步得到简化。
- 设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得：

$$T(S,t)=a_i+T(S-\{i\},b_i+\max\{t-a_i,0\})=a_i+a_j+T(S-\{i,j\},t_{ij})$$

- 其中，

$$\begin{aligned}t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\&= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\&= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\&= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}\end{aligned}$$

- 如果作业 i 和 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则称作业 i 和 j 满足 Johnson 不等式。

4.3.7 流水线调度问题—Johnson 不等式

- 交换作业 i 和作业 j 的加工顺序，得到作业集 S 的另一调度，它所需的加工时间为 $T'(S,t)=a_i+a_j+T(S-\{i,j\},t_{ji})$

- 其中， $t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$

- 当作业 i 和 j 满足 Johnson 不等式时，有

$$\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$$

$$a_i + a_j + \max\{-b_i, -a_j\} \leq a_i + a_j + \max\{-b_j, -a_i\}$$

$$\max\{a_i + a_j - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$$

- 由此可知 $\max\{t, a_i + a_j - b_i, a_i\} \leq \max\{t, a_i + a_j - b_j, a_j\}$ ，交换它们的加工顺序后，不增加加工时间。对于流水作业调度问题，必存在最优调度，使得作业 (i) 和 $(i+1)$ 满足 Johnson 不等式。进一步还可以证明，调度满足 Johnson 法则当且仅当对任意 $i < j$ 有

- 由此可知，所 $\min\{b_{\pi(i)}, a_{\pi(j)}\} \geq \min\{b_{\pi(j)}, a_{\pi(i)}\}$ 均为最优调度。

4.3.7 流水线调度问题—算法分析

- 流水作业调度问题的 Johnson 算法：
 - 令 $N_1 = \{i \mid a_i < b_i\}$, $N_2 = \{i \mid a_i \geq b_i\}$;
 - 将 N_1 中作业依 a_i 的非减序排序; 将 N_2 中作业依 b_i 的非增序排序;
 - N_1 中作业接 N_2 中作业构成满足 Johnson 法则的最优调度。
- **思考**: 上述步骤得到的作业序列满足 Johnson 法则。
- 算法复杂度分析: 算法的主要计算时间花在对作业集的排序。因此, 在最坏情况下算法所需的计算时间为 $O(n \log n)$ 。所需的空间为 $O(n)$ 。